STUDENT MANAGEMENT SYSTEM

Simple Web-Based Student Information and CRUD Management System

Mini Project – 1 Credit

1. Abstract

The Student Management System is a simple web-based application designed to help educational institutions and administrators manage student records efficiently. The system allows users to perform complete CRUD (Create, Read, Update, Delete) operations on student details including registration, profile search, information updates, and record removal. The application uses Streamlit for an interactive user interface and FastAPI for the backend REST API, with SQLite used to store student records persistently. The project demonstrates how a lightweight frontend and backend can work together to provide a practical, service-oriented academic management solution.

2. Problem Statement

Educational institutions and department staff often experience difficulties maintaining student records through traditional paper registers or unstructured spreadsheets, leading to errors, duplicate entries, data loss, and slow lookups. A simple digital management assistant can provide centralized data entry, instant search by student ID, unique email validation, and streamlined updates before more complex ERP systems are implemented.

3. Objectives

- Develop a simple web application for streamlined student record management.
- Allow users to register new students with name, age, department, and email address.
- Provide complete CRUD functionality to view, search, modify, and delete student records.
- Implement automated validation to ensure positive IDs, valid age boundaries, and unique emails.
- Use FastAPI to provide backend REST APIs and Streamlit to create the interactive frontend.
- Store student details in SQLite database using SQLAlchemy ORM for reliable local persistence.

4. Technologies Used

Technology	Purpose
Python 3	Core programming language and application business logic
FastAPI	Backend REST API framework and request/response handling
Streamlit	Interactive web frontend for user interaction
SQLite	Storage of student records (students.db)
SQLAlchemy	Object Relational Mapping (ORM) and database session management
Pydantic	Validation of API request and response data schemas
Requests	HTTP communication between Streamlit frontend and FastAPI backend
Uvicorn	Asynchronous ASGI server running the FastAPI backend

5. System Architecture

The application follows a simple three-layer flow. The user interacts with the Streamlit frontend. Streamlit sends the student management request to the FastAPI backend through HTTP. FastAPI validates the payload against Pydantic schemas and passes the request to the CRUD database layer, which interacts with the SQLite database via SQLAlchemy ORM. The result is returned to Streamlit and displayed to the user. All student records are stored in SQLite.

User → Streamlit Frontend → FastAPI Backend → CRUD Logic / SQLAlchemy → SQLite

6. Main Features

- Home dashboard with student summary statistics and system overview.
- Student registration form capturing name, age, department, and email address.
- Real-time duplicate email validation preventing duplicate student admissions.
- Interactive student directory displaying all enrolled students in a clean table.
- Targeted student search by entering a unique Student ID.
- Student detail modification form for updating name, age, department, or email.
- Safe deletion module to remove obsolete records with confirmation.
- FastAPI Swagger interactive documentation available through /docs.
- Backend root health-check endpoint confirming API server availability.
- Modern, responsive user interface with clear success and error notifications.

7. Database Schema & Core Entities

The system maintains a relational database model for student records. Each record represents an enrolled student with essential academic and personal details:

- **Student ID:** Unique auto-incrementing integer identifier (Primary Key).
- **Student Name:** String field representing the full legal name of the student.
- **Student Age:** Positive integer representing the student's age (15 to 100).
- **Department:** String indicating academic branch (e.g., Computer Science, Mechanical, ECE).
- **Email Address:** Unique indexed string ensuring no two students share the same email.

Field	Data Type	Constraints	Description
id	Integer	Primary Key, Auto-increment	Unique student ID number
name	String(100)	Not Null	Full legal name of the student
age	Integer	Not Null (15–100)	Current age of the student
department	String(100)	Not Null	Academic department / branch
email	String(100)	Unique, Indexed	Student institutional email address

8. FastAPI Endpoints

Method	Endpoint	Purpose
GET	/api/health or /	Checks whether the backend server is running
GET	/students	Retrieves list of all enrolled students from the database
GET	/students/{id}	Retrieves a single student's details by their unique ID
POST	/students	Creates a new student record after validating unique email
PUT	/students/{id}	Updates an existing student's name, age, department, or email
DELETE	/students/{id}	Deletes a student record permanently by ID

9. Working of the System

1. The user launches the Streamlit frontend and FastAPI backend applications.
2. The user opens the Streamlit web dashboard in their browser.
3. To add a student, the user fills in the name, age, department, and email, then clicks Submit.
4. Streamlit transmits the payload via an HTTP POST request to the FastAPI /students endpoint.
5. FastAPI validates data types with Pydantic and checks SQLite to ensure the email is not already registered.
6. The CRUD module inserts the student into SQLite via SQLAlchemy and generates a unique ID.
7. The updated student profile is returned as a JSON response and rendered as a confirmation in Streamlit.
8. The user can view all students in the interactive directory, look up records by ID, update fields, or delete entries.

10. Testing and Results

The application was thoroughly verified through the Streamlit interface and FastAPI Swagger documentation. The backend health endpoint responded successfully, all CRUD endpoints processed requests accurately, and student records were successfully stored and managed in SQLite.

Test Case	Expected Result	Status
Backend health check	API reports root welcome and running status	**Passed**
Student enrollment	New student added with auto-increment ID and HTTP 201	**Passed**
Duplicate email check	HTTP 400 Bad Request returned with error message	**Passed**
Retrieve student list	All active records displayed in tabular format	**Passed**
Search student by ID	Student record returned; HTTP 404 for nonexistent ID	**Passed**
Update student record	Student fields updated and persisted in SQLite	**Passed**
Delete student record	Record deleted; success message returned	**Passed**
Swagger documentation	API endpoints and schemas interactive at /docs	**Passed**

11. Advantages

- Simple and easy-to-use graphical interface with zero training required.
- Fast response times and high performance powered by FastAPI.
- Guaranteed data integrity with Pydantic validation and unique email constraints.
- Lightweight technologies perfectly suited for an academic mini project.
- Zero-configuration persistence with SQLite requiring no database server setup.
- Modular codebase making it straightforward to add more academic modules in the future.

12. Limitations

The application provides a centralized student record management system designed for single-node academic use. It does not include role-based multi-user authentication (Admin, Teacher, Student roles) or fine-grained access control. Additionally, it currently does not support bulk CSV file uploads for batch student enrollment.

13. Future Enhancements

- Add user authentication and role-based permissions (Admin, Faculty, Student).
- Implement bulk student import and export using Excel / CSV spreadsheets.
- Introduce course enrollment, semester grades, and attendance tracking modules.
- Add automated PDF report card and student ID card generation.
- Migrate SQLite to cloud PostgreSQL for large-scale multi-institution deployment.

14. Conclusion

The Student Management System successfully demonstrates a complete full-stack web application using Streamlit and FastAPI. It accepts student registration details, validates them with strict schema rules, performs complete CRUD operations via SQLAlchemy, and stores records securely in SQLite. The project provides a practical example of REST API communication, modern frontend development, backend request handling, and relational database management in a compact academic application.